Canonical CBOR snapshots carried by `ir_stream` — implementation plan

Claude (core_cpp agent), working in hymeko/core

Created-at: 2026-09-15 21:13 CEST

Authorisation

```
APPROVED-CORE-EDIT: ir-stream-cbor-snapshot
```

The user supplied this token verbatim in the core_cpp Claude Code chat on 2026-09-15 (turn after 21:07 CEST), after two decisions in the same chat: the Core extension is wanted ("Core-bővítés is"), the Rust Core change goes on its own branch ("a rust core IR külön branchen menjen"), and the complete IR snapshot stays canonical CBOR ("CBOR nem messagepack"). The token covers exactly the boundary in Section 3; the commit message, if the user later authorises one, repeats it in a footer. No commit or push is authorised by this plan.

Git state recorded before the plan: `git fetch origin` succeeded at 21:03 CEST; `origin/main` = `368c7fda59cc1ae4834d41581841778e57d677ba`. The existing checkout `hymeko_framework/core` is on `perf/compiler-repeated-work` with another worker's uncommitted `ir-value-updates` edits (`execution.rs`, `execution/value_updates.rs`); it is not switched or touched. **Worktree exception**: reason — the only checkout carries another worker's dirty branch; path `D:/hakiko_ai_ws/03_implementation/worktrees/core-ir-stream-cbor-snapshot`; branch `feat/ir-stream-cbor-snapshot` from `origin/main` (upstream unset so no push can target main); owner — this core_cpp agent; retirement — after merge or abandonment, inspect for dirty files and unpublished commits before removal.

ETA

About 2.5 hours, completion near 2026-09-15 23:45 CEST. Basis: plan and PDF 30 min; section kind and CBOR stream bridge with unit tests 50 min; integration test 20 min; first cold build of `hymeko_core` in the new worktree plus targeted tests 25 min (`-j 8`); fmt, clippy, symbols, duplication 15 min; report and archive 15 min.

1 Goal and ownership

A complete, validated HyMeKo IR must travel over the existing `ir_stream` transaction protocol (`SnapshotBegin` → `SnapshotChunk`* → `Commit`) in fragments small enough for a UART link and large enough for a LAN, and a consumer must turn a committed transaction into a Core-validated IR without source parsing. Two consumers need this now: the robotics HIVE store over Zenoh (another agent, `robotics/docs/plans/2026-09-12-belief-planner/plan.md`, "LAN HIVE implementation") and, later, the core_cpp MCU runtime over the ST-LINK virtual COM port.

Ownership, verified against code: `hymeko_core::ir_stream` owns the wire vocabulary, validation and transaction admission; its assembler explicitly does not apply transactions. `writers::cbor_writer` is "the only owner of the CBOR schema" and already depends on `ir_stream::DecodeLimits`. Today no section kind can carry declarations, hierarchy, symbols or imports (`section.rs` defines ten numeric kinds; `ElementKind::Raw8` exists for opaque bodies).

This plan therefore adds one opaque section kind to `ir_stream` and the CBOR-specific producer and consumer to the CBOR owner. It adds no second codec, no dependency and no IR schema.

2 Discovery ledger

```
DISCOVERY LEDGER - canonical-CBOR snapshot carriage over ir_stream
  find:    find hymeko_core/src/{writers,ir_stream} hymeko_core/tests
           -iname "*snapshot*" -o -iname "*stream*"
           -> 2 hits: src/ir_stream (dir), tests/ir_stream_admission.rs
  grep:    grep -rnE "CanonicalIr|cbor[_ ]snapshot|snapshot[_ ]cbor|IrCbor"
           hymeko_core/src hymeko_core/tests docs -> 0 hits
           grep -rlE SnapshotChunk hymeko_core/{src,tests,benches} -> 4 hits:
           src/ir_stream/{assembler,codec,envelope}.rs, benches/ir_stream_bench.rs
           (vocabulary and tests; no CBOR producer)
           axon, hive, robotics *.rs: SnapshotBegin|encode_message|CanonicalIr|
           ir_stream -> 0 hits; "SectionKind::" outside core -> 0 hits
  symbols: SYMBOLS.md -> CborPayload, DecodedCborPayload, CborError;
           CommittedTransaction, StagedSection, TransactionAssembler,
           EnvelopeDraft, MessageType, SectionEntry, ElementKind, DecodeLimits
  read:    ir_stream/{mod,section,codec,limits,assembler,envelope,version}.rs,
           writers/cbor_writer.rs, cbor_writer/tests.rs, tests/mod.rs,
           tests/ir_stream_admission.rs, tests/unified_ir/validation/
           cbor_roundtrip.rs, docs/hypergraph-synchronization.md 4-6,
           docs/plans/2026-09-04-canonical-cbor-codec, robotics LAN HIVE plan
  verdict: EXTEND src/ir_stream/section.rs + NEW
           src/writers/cbor_writer/stream.rs; rejected the bridge inside
           ir_stream (transport vocabulary must not depend on the CBOR owner)
           and a MessagePack writers/snapshot.rs (user: CBOR, not MessagePack)

DISCOVERY LEDGER - tests/ir_stream_cbor_snapshot.rs
  verdict: NEW, wired through tests/mod.rs; rejected ir_stream_admission.rs
           (charter: admission with empty tables, not installation) and
           cbor_roundtrip.rs (CBOR facade only; carries memory probes)

DISCOVERY LEDGER - docs/plans/2026-09-15-ir-stream-cbor-snapshot
  find:    ls docs/plans | grep -iE "stream|cbor|snapshot" -> 2 hits:
           2026-08-28-ir-stream-wire-codec, 2026-09-04-canonical-cbor-codec
  verdict: NEW; both hits are completed phases under other approval slugs
```

3 Approved boundary (CORE.YAML items touched)

Path	Change	Authority
`hymeko_core/src/ir_stream/section.rs`	variant `SectionKind::CanonicalIrCbor` = wire 11, mapping, rustdoc, unit tests	approved core
`hymeko_core/src/writers/cbor_writer.rs`	`mod stream;` and re-exports	approved core
`hymeko_core/src/writers/cbor_writer/stream.rs` (new)	producer, consumer, error type	approved core
`hymeko_core/src/writers/cbor_writer/tests.rs`	unit tests reusing the existing fixture	approved core
`CORE.YAML`	`approved_edits` record of this slug	approved core

hymeko_core/tests/ir_stream_cbor_snapshot.rs (new), tests/mod.rs	integration test	allowlisted
docs/hypergraph-synchronization.md §5.3	table row; implemented versus specified kinds	not core
SYMBOLS.md	regenerated	generated
docs/plans/**, reports/**	traceability	allowlisted

Out of scope: MessagePack or any new codec; dependency or Cargo.toml/Cargo.lock changes; IR schema, canonical hash, CBOR wire bytes; the envelope, section validator and assembler logic (only the kind mapping grows); installation into ExecutionState/CompiledModel (execution.rs, the HIVE agent's ir-value-updates area); delta semantics for IR structure; authenticity (the codec documents it as a transport concern); the C consumer in core_cpp; commit, push, merge.

4 Interface changes

```
// hymeko::ir_stream::section
pub enum SectionKind { ..., CanonicalIrCbor, Unknown(u16) }  // wire 11, critical,
                                                             // repeatable, not skippable
// hymeko::writers::cbor_writer (re-exported from the child module `stream`)
pub struct SnapshotMessageDraft {
    pub stream_id: u64, pub first_seq: u64, pub txn_id: u64,
    pub base_version: SnapshotVersion, pub target_version: SnapshotVersion,
    pub max_fragment_bytes: usize,
}
pub fn encode_snapshot_messages(cbor: &[u8], draft: &SnapshotMessageDraft)
    -> Result<Vec<Vec<u8>>, CborStreamError>;
pub fn decode_snapshot(transaction: &CommittedTransaction, limits: &DecodeLimits)
    -> Result<DecodedCborPayload, CborStreamError>;
pub enum CborStreamError {
    EmptyDocument, ZeroFragmentSize, SequenceOverflow,
    NotASnapshot { kind: TransactionKind },
    NoFragments,
    UnexpectedSection { index: usize, kind: SectionKind },
    WrongElementKind { index: usize, got: ElementKind },
    TooLarge { got: u64, max: u64 },
    Cbor(CborError),
}
```

5 Contracts

Wire. CanonicalIrCbor sections carry ElementKind::Raw8 bodies whose concatenation, in staging order (message order, then table order), is one canonical CBOR document of hymeko/model/ir/cbor/v1. The kind is below CRITICAL_SECTION_MAX, so an older consumer refuses it with UnknownMandatorySection instead of skipping meaning. It is repeatable, so one document may span any number of chunks; existing numeric kinds keep their singleton and patch rules unchanged.

Producer. Preconditions: cbor is non-empty and was produced by encode or encode_compiled; max_fragment_bytes > 0; first_seq is the stream's next sequence number; target_version obeys the assembler's rules (a topology bump also bumps weights and features). Postcondition: messages SnapshotBegin (empty section table), one SnapshotChunk per fragment with a single CanonicalIrCbor section and FLAG_LAST_CHUNK on the last, then Commit; contiguous sequence numbers checked for overflow; every message passes validate_message under limits that admit one fragment (max_section_bytes $\geq$ fragment, max_payload_bytes $\geq$ 40+ fragment). The producer does not decode the document; the consumer is the validation authority.

Consumer. Preconditions: `transaction` came from `TransactionAssembler::apply`; the caller's `AssemblerLimits` admit the document (`max_staged_bytes`, `max_staged_sections`, `max_chunks`). Rules, in order: the kind is `Snapshot`; at least one section; every section is `CanonicalIrCbor` (a CBOR snapshot is the complete state, so a numeric section beside it would be a second authority) with `Raw8` elements; the checked total length is at most `limits.max_message_bytes`, tested before the concatenation buffer is allocated; then `cbor_writer::decode` with the same limits. Postcondition: the returned `DecodedCborPayload` satisfies every existing CBOR boundary check (preflight, schema, IDs, index, coherence, recomputed canonical hash). Failure leaves nothing published; this function holds no state.

Invariants. Envelope validation, section-table validation, digest coverage, replay and version admission are unchanged. `decode(encode(x))` identity carries over: a document sent in any fragmentation decodes to the same canonical hash as a direct `decode`.

6 Test strategy (correctness only)

1. **Unit,** `section.rs`: wire 11 round-trips; the kind is neither singleton nor skippable; a table with two `CanonicalIrCbor` sections validates; values 12–999 still reject as unknown mandatory.

2. **Unit,** `cbor_writer/tests.rs` (existing one-node fixture, real `validate_message` and `TransactionAssembler`): fragment sizes 1, 3, exact length and larger than the document round-trip to the direct-decode canonical hash; message count, flags, contiguous sequence and per-message validation. Negative controls: empty document, zero fragment size, sequence overflow, a `Delta` transaction, a snapshot without sections, a mixed numeric section, a non-`Raw8` section, a total above `max_message_bytes`, a dropped middle fragment and two swapped fragments (both must fail `decode`).

3. **Integration,** `tests/ir_stream_cbor_snapshot.rs`: every lowerable corpus fixture (`require_fixtures`, fail-closed) is encoded with `encode_compiled`, split into 4096-byte fragments, driven through the public wire validator and assembler, and decoded; canonical hash, root path, imports and declaration count equal the direct decode. One fixture repeats with fragment sizes 1 and 257.

4. **Regression**: `cargo test -p hymeko_core -lib ir_stream`, `-lib writers::cbor_writer`, `-test mod ir_stream_admission`, and the functional `cbor_roundtrip` tests selected by exact name after inspection. The `canonical_cbor_memory_*` evidence probes and every benchmark stay unrun.

5. **Static gates**: `cargo fmt -all - -check`; `cargo clippy -p hymeko_core -all-targets - -D warnings`; `contract-symbols -root . -check` after regeneration; `contract-duplication -root .` against the baseline.

Every test selection is inspected first: the aggregate `mod` test crate also contains opt-in profile modules, so only filtered test names run.

7 Performance budget

No benchmark, profile or timing measurement is authorised or run. Resource bounds are structural: the consumer allocates one buffer of the checked document length (at most `DecodeLimits::max_message_bytes`, default 256 MiB) in addition to the assembler's staged fragments, so peak memory is about twice the document plus the decoded IR. The producer allocates one `Vec<u8>` per message ($144 + 40+$ fragment bytes). Build jobs are limited to 8 to stay far below the 16 GB RSS cap, which Windows cannot enforce with `systemd-run`.

8 Rollback

All changes stay uncommitted on `feat/ir-stream-cbor-snapshot`. Rollback is `git restore` and deletion of the new files in the worktree, or removal of the branch and worktree after inspection. Wire value 11 is unpublished until a merge, so no persisted message or fixture depends on it.

9 Risk anticipation

- **Performance-contract preservation.** The section validator, envelope, digest, assembler limits and replay window keep every existing check; the only validator change is one more accepted kind below the critical boundary. Staging limits still bound the fragment count and staged bytes; the consumer adds a length check before its single allocation, so the new path inherits the "limits before allocation" contract.
- **Worst-case input.** A hostile or buggy stream sends many 1-byte fragments (bounded by `max_chunks`/`max_staged_sections`), fragments whose sum exceeds the limit (rejected before allocation), reordered or missing fragments (rejected by CBOR preflight, validation or canonical hash), or valid CBOR of a different model (accepted as that model: authenticity is a transport concern and stays documented as such). Production documents are robot and planner models from kilobytes to megabytes; the default limits leave headroom.
- **Wire-number collision.** Another unpublished branch could claim kind 11. Remote branches show no `ir_stream` work; the concurrent `ir-value-updates` edits touch `execution.rs` only.
- **Exhaustive matches.** Adding a public enum variant breaks exhaustive matches in dependants. No crate outside core in the workspace matches on `SectionKind`.
- **Version rule surprise.** Producers that bump topology alone are rejected by the assembler; the producer rustdoc states the rule and a test drives it.
- **Memory of large snapshots.** Concatenation doubles the document transiently; a streaming decoder is not possible without changing the CBOR preflight, which is out of scope.
- **Scope creep into the HIVE agent's area.** No edit to `module_store`; the hand-off point is the validated `DecodedCborPayload`.

10 Tooling notes

`mmdc` is absent on this host (`tools.yaml`); `plan.mmd` is source only. Clippy runs for `hymeko_core` with all targets instead of the whole workspace, because `hymeko_wgpu` is untouched. The contract gates run through the installed entry points pinned by `tools/hooks` (provider `8871abe`).

Architecture

